

/home/dan/TMP — harness + tests, step by step

Reproducible log of building the µOS++ test harness in /home/dan/TMP, with the tests of the six repositories wired into the tests paradigm, named qemu (runs under QEMU) or hwd (runs on hardware), and runnable from the VS Code xPack plugin.

Everything below uses \$HOME and \$PWD only — no hard-coded user — so it reproduces anywhere on this machine.

what	path
bare mirrors (clone source)	\$HOME/Downloads/GIT/micro-os-plus-iii-*.git
this workspace	\$HOME/TMP
node / npm / xpm	node v25.8.2, npm 11.11.1, xpm 0.23.2

Step 0 — the workspace

```
mkdir -p "$HOME/TMP"
cd "$HOME/TMP"
```

Step 1 — clone the six repositories

The harness lives in micro-os-plus-iii-smp.git/tests/, and it finds the ports as siblings named micro-os-plus-iii-aarch32.git and ...-aarch64.git (see tests/cmake/tests-main.cmake), so the clones keep the .git suffix.

```
cd "$HOME/TMP"
for r in aarch32 aarch64 cortexm devices posix-arch smp; do
  git clone "$HOME/Downloads/GIT/micro-os-plus-iii-$r.git" \
    "$HOME/TMP/micro-os-plus-iii-$r.git"
done
```

Step 2 — sibling naming (no symlinks)

Two naming conventions meet here:

- the **harness** looks for the ports **with** the suffix (`../../micro-os-plus-iii-aarch32.git`);
- a **port** looks for its dependencies by the plain name (`../micro-os-plus-iii-smp`, `../micro-os-plus-iii-devices`).

The ports now accept **either**: if the plain-name sibling is absent and the `*.git` one is present, they use it — a `.git` fallback in each port's `CMakeLists.txt` (`UOS_SMP_DIR` / `UOS_DEVICES_DIR`) and in the port test scripts' `SMP_DIR`. So the workspace keeps **only** the `.git` clones and needs no suffix-less symlinks:

```
cd "$HOME/TMP"
ls -d micro-os-plus-iii-*.git
```

The harness itself already used the `.git` paths, so nothing else changes.

Step 3 — the harness tools

```
cd "$HOME/TMP/micro-os-plus-iii-smp.git/tests"
npm install      # node_modules
xpm install      # xpacks/ symlinks (cmake, ninja, build-helper,
                  validator, chan-fatfs)
```

Step 4 — one configuration, end to end

Every configuration is driven by four `xpm run` actions, in this order: `install`, `prepare`, `build`, `test`. They are the `cmake-actions` in `tests/package.json`, and its properties hold the exact commands:

action	runs	which is
install	<code>xpm install --config <config></code> <code>cmake -S . -B build/<config> -G Ninja -D CMAKE_BUILD_TYPE=... -D PLATFORM_NAME=... --log-level=VERBOSE -D CMAKE_TOOLCHAIN_FILE=xpacks/@micro-os-plus/build-helper/cmake/toolchains/<file></code>	the pinned toolchain into <code>build/<config>/xpacks/.bin</code>
prepare	<code>commandCMakePrepareWithToolchain</code>	
build	the same <code>cmake ...</code> without the toolchain file, then <code>cmake --build build/<config></code>	<code>commandCMakeReconfigure</code> + <code>commandCMakeBuild</code>

action	runs	which is
test	cd build/<config> && ctest -V (each config narrows this to ctest -V -LE hwd)	commandCMakePerformTests
clean	cmake --build build/<config> --target clean	commandCMakeClean

Three rules follow, and each has bitten:

- **install before prepare.** prepare names a toolchain file that lives *inside* the build folder (build/<config>/xpacks/...); before install it is not there and the configure fails.
- **prepare before build.** Only prepare passes -D CMAKE_TOOLCHAIN_FILE=.... The build action's reconfigure does **not**, so a build folder that was never prepared configures with the host compiler — or the wrong cross compiler. (prepare is idempotent; the build reconfigure keeps the cached toolchain, so once prepared, builds are fine.)
- **test does not build.** xpm run test on a folder that was only configured reports every case as failed, because the executables were never linked:

```
no *-host executables in .../platform-bin/port-tests
```

Run build first. The per-suite actions are the exception — each is self-contained (commandCMakePrepareWithToolchain && commandCMakeBuild && ctest -V -R <case>), so xpm run test-rtos-apis-host --config native-cmake-gcc-debug builds and runs that one case on its own. That is why the VS Code plugin's per-test actions work on a cold tree while the generic test does not.

The pinned toolchain is installed **inside the build folder** (build/<config>/xpacks/.bin), so build/<config>/ must not be deleted. If the cache is ever poisoned (e.g. by a configure that ran before install), delete only the cache, not the whole folder:

```
cd "$HOME/TMP/micro-os-plus-iii-smp.git/tests"
xpm run install --config aarch32-rpi-zero-2w-cmake-gcc-debug
xpm run prepare --config aarch32-rpi-zero-2w-cmake-gcc-debug
xpm run build   --config aarch32-rpi-zero-2w-cmake-gcc-debug
xpm run test    --config aarch32-rpi-zero-2w-cmake-gcc-debug
```

Debug and release are separate build folders

-release is not a mode of the debug folder. Every configuration is its own build/<config>/ — there are 62 of them (31 debug + 31 release) — and a release config inherits the debug config's *properties* but **not** its build folder, so it needs its own install. Skip that and prepare fails as soon as the dependencies are added:

```
CMake Error at xpacks/@micro-os-plus/build-helper/cmake/micro-os-plus-build-helper.cmake:112 (message):
```

```
Missing ../build/native-cmake-clang17-release/xpacks/@xpack-3rd-party/libucontext/CMakeLists.txt
```

native-dependencies brings in libucontext (and posix-arch), so that folder has to exist before the configure; it is empty when install was never run. The clang configs also pin their own compiler — native-cmake-clang17-release wants clang 17.0.6-3.1, not whatever clang is on PATH.

All the release configurations

The emulated/host set, release, four steps each:

```
cd "$HOME/TMP/micro-os-plus-iii-smp.git/tests"
for C in \
  native-cmake-gcc-release \
  qemu-cortex-m0-cmake-gcc-release qemu-cortex-m3-cmake-gcc-
    release \
  qemu-cortex-m4f-cmake-gcc-release qemu-cortex-m7f-cmake-gcc-
    release \
  cortexm-pico2-cmake-gcc-release \
  cortexm-pico2-rp2350b-psram-cmake-gcc-release \
  aarch32-rpi-zero-2w-cmake-gcc-release aarch64-rpi-zero-2w-cmake-
    gcc-release ; do
  xpm run install --config "$C" && xpm run prepare --config "$C"
  && \
  xpm run build --config "$C" && xpm run test --config "$C"
done
```

The hardware set, release, build only — their cases are all hwd:

```
for C in \
  cortexm-pico2-pizero-cmake-gcc-release \
  cortexm-nucleof411-cmake-gcc-release cortexm-weactf411-cmake-
    gcc-release \
  cortexm-weactf412-cmake-gcc-release \
  nucleo-f411re-cmake-gcc-release nucleo-f767zi-cmake-gcc-release
  \
  nucleo-h743zi-cmake-gcc-release raspberrypi-pico-cmake-gcc-
    release ; do
  xpm run install --config "$C" && xpm run prepare --config "$C"
  && \
```

```
xpm run build --config "$C"  
done
```

The host toolchain variants are the same recipe with a different toolchain file: native-cmake-
{gcc,gcc11,gcc12,gcc13,gcc14,clang,clang13,clang14,clang15,clang16,clang17,clang18,clang19,sys}-release. aarch32-luckfox-lyra-cmake-gcc-release builds its all-hwd set.

Step 5 — put all the port's tests into the paradigm

The port owns its tests (micro-os-plus-iii-aarch32.git/test/rpi-zero-2w/<app>/) and its own builder (test/CMakeLists.txt), which compiles each test twice — <app>-qemu and <app>-hwd. The port adds that builder only when it is the top-level project (PROJECT_IS_TOP_LEVEL), so the harness normally re-lists a few tests by hand.

Instead, the platform now **reuses the port's builder** and registers what it produced, so every test the port has appears, and a test added to the port appears with no edit. In tests/platforms/aarch32-rpi-zero-2w/CMakeLists.txt:

```
include ("cmake/platform-library.cmake")  
  
# The port publishes most board facts as CACHE INTERNAL, but not  
# UOS_BOARD_DEVICES; read the board declaration so the builder  
# sees it.  
include ("${UOS_AARCH32_DIR}/test/boards/${BOARD}/board.cmake")  
  
# Build every port test (qemu + hwd) by reusing the port's  
# builder.  
add_subdirectory ("${UOS_AARCH32_DIR}/test" "port-tests")  
  
set (_port_bin "${CMAKE_CURRENT_BINARY_DIR}/port-tests")  
set (_shim_img "${_port_bin}/shim8.img")  
    # the builder's QEMU boot shim  
set (_runner "${CMAKE_SOURCE_DIR}/../test_smpl")  
  
# One CTest case per (test, variant): <platform>-<app>-qemu /  
# -hwd.  
file (GLOB _test_dirs LIST_DIRECTORIES true "${_port_test_dir}/*")  
foreach (_dir IN LISTS _test_dirs)  
    ... # skip include/ and src/, keep the directories that hold  
    # sources  
    add_test (NAME "${PLATFORM_NAME}-${_app}-qemu"  
        COMMAND bash "${_runner}/run-qemu.sh" "${_port_bin}"  
        "${_qemu}" -M raspi3b -smp 4)  
    set_tests_properties ("${PLATFORM_NAME}-${_app}-qemu" PROPERTIES
```

```

ENVIRONMENT "UOS_QEMU_SHIM=${
    {_shim_img};UOS_QEMU_LOAD_ADDR=0x10000;UOS_QEMU_ONLY=${
    {_app}}"
LABELS qemu)
add_test (NAME "${PLATFORM_NAME}-${_app}-hwd"
    COMMAND bash "${_runner}/run-hw.sh" "${_port_bin}" "$
    {_app}" 600)
set_tests_properties ("${PLATFORM_NAME}-${_app}-hwd" PROPERTIES
    ENVIRONMENT "BUILD=${_port_bin}" LABELS hwd)
endforeach ()

```

The run side is the **shared runner** (micro-os-plus-iii-smp.git/test_smpl/) that the port's own qemu.sh/hw.sh call — run-qemu.sh greps the RESULT: line, run-hw.sh drives OpenOCD and takes one test (one per power cycle).

Step 6 — add the harness suites to the platform

A harness suite supplies no main()/hooks, so it links micro-os-plus::platform-support. It ends through the platform's strong _Exit() (SYS_EXIT), so QEMU returns an exit code and there is no RESULT line to grep — the direct QEMU command, not run-qemu.sh.

```

function (add_harness_suite _suite _name)
    add_executable (${_name})
    target_link_libraries (${_name} PRIVATE micro-os-plus::common-
        options
        test::${_suite} micro-os-plus::iii micro-os-plus::platform
        micro-os-plus::platform-support ${T_LIBRARIES})
    set_target_properties (${_name} PROPERTIES OUTPUT_NAME "$
        {_name}" SUFFIX "")
    add_test (NAME "${PLATFORM_NAME}-${_name}-qemu"
        COMMAND "${_qemu}" --machine raspi3b -smp 4 --
        nographic --serial none
            --semihosting-config enable=on,target=native
            --kernel "${_shim_img}"
            --device "loader,file=$<TARGET_FILE:${_name}
            >.bin,addr=0x10000")
endfunction ()

add_harness_suite (mutex-stress mutex-stress-test)

```

Step 7 — verify

```

cd "$HOME/TMP/micro-os-plus-iii-smp.git/tests"
B=build/aarch32-rpi-zero-2w-cmake-gcc-debug
PATH="$PWD/$B/xpacks/.bin:$HOME/.local/xPacks/@xpack-dev-tools/
    qemu-arm/9.2.4-1.1/.content/bin:$PATH" \

```

```
ctest --test-dir "$B" -N # list
PATH=... ctest --test-dir "$B" -R "smp_test0-qemu|mutex-stress" -V
```

Result: **26 CTest cases** — the port's tests as qemu/hwd (14 qemu + 12 hwd), plus the mutex-stress and rtos-apis suites. Verified passing: smp_test0-qemu, smp_test1-qemu, smp_test3-qemu, mutex-stress-test-qemu, rtos-apis-test-qemu.

Status, and what is still to do

Done

- the workspace, the six clones (.git only — no suffix-less symlinks; the ports accept either name);
- the harness tools, and the configurations end to end;
- platforms/aarch32-rpi-zero-2w — **all** the port's tests as <platform>-<app>-qemu / -hwd, plus mutex-stress and rtos-apis (**26** cases: 14 qemu + 12 hwd). Verified passing: smp_test0/1/3-qemu, mutex-stress-test-qemu, rtos-apis-test-qemu.
- platforms/aarch32-luckfox-lyra — the port's 19 tests + mutex-stress, **-hwd only** (the board is hardware-only); 20 CTest cases, never run by a plain ctest.
- platforms/aarch64-rpi-zero-2w — the port's tests × qemu/hwd, plus mutex-stress and rtos-apis (**26** cases: 14 qemu + 12 hwd); verified passing: smp_test0-qemu, mutex-stress-test-qemu, rtos-apis-test-qemu. The _Exit() link issue is **fixed**: the port's own src/handlers.cpp already defines a strong semihosting _Exit/_exit, so the platform-support's copy was a duplicate definition; it was removed and the suite enabled.
- platforms/native — the **POSIX-arch** port on the **host compiler** (micro-os-plus-iii-posix-arch.git), selected by the new ^native branch in tests-main.cmake; 12 host tests (the port's 11 + cmsis-os-validator), verified passing: smp_test0-host, cmsis-os-validator-host.

The remaining harness suites — now wired and passing.

- rtos-apis — **now wired and passing on both AArch32 and AArch64** (...-rtos-apis-test-qemu, ~7 s each). Three things were wrong:
 1. **lifo::do_allocate bug (kernel)**. It set chunk = free_list_ and, when that head chunk was smaller than the request (rem < 0), never cleared chunk — so the loop broke and handed the too-small chunk to internal_align_, which asserts. first_fit_top advances chunk = chunk->next and is correct; lifo now sets chunk = nullptr on the too-small path, so it falls through to the out-of-memory handler (which may return to coalesce) or returns nullptr.
 2. **Undersized RTOS dynamic memory**. sources/rtos-apis/.../os-app-config.h set

OS_INTEGER_RTOS_DYNAMIC_MEMORY_SIZE_BYTES to 14 KiB, but the default resource is that system heap, and FatFs asks for up to MAX_MALLOC (0x8000) — with the out-of-memory hook fatal (no exceptions), the test aborted. Raised to 512 KiB.

3. **AArch64 newlib signature clash.** aarch64-none-elf declares read/write with `_READ_WRITE_RETURN_TYPE (int)`, the kernel header used `ssize_t`. `c-syscalls-aliases-standard.h` now uses `_READ_WRITE_RETURN_TYPE` when the system header provides it. The AArch64 platform also defines `OS_USE_SEMIHOSTING_SYSCALLS` so the tests skip their raw-POSIX-C-API sub-tests (this port does not carry that layer).
- **cmsis-os-validator, blinky, instrumentation — now build.** The nucleo-f411re platform (the upstream Cortex-M one, already in the tree) wires all three, plus `rtos-apis` and `mutex-stress`. They did not build before only because the SEGGER xpacks were missing:
 - `xpm install --config nucleo-f411re-cmake-gcc-debug` now installs `@xpack-segger/rtt@8.56.1-2` and `@xpack-segger/system-view@3.60.5-2` from `github:xpack-3rd-party/...`; the platform then builds `rtos-apis-test.elf`, `rtos-apis-instrumentation-test.elf`, `mutex-stress-test.elf`, `cmsis-os-validator-test.elf`, `blinky-test.elf` and `instrumentation-test.elf`.
 - `cmsis-os-validator` needs the NVIC API, which the nucleo-f411re platform-support provides (CMSIS core); `blinky` gets its `blink-led.h` from the platform (PA5), and `instrumentation` links `segger::rtt + segger::system-view`.
 - All four of its `add_tests` drive the board through OpenOCD, so they are now labelled `hwd` (a directory-level `set_tests_properties`); the generic test action skips them, and `test-nucleo-f411re-cmake` runs them on a bench (`ctest -v`).
- **Native cmsis-os-validator — now passing 60/60** (native-`cmsis-os-validator-host`, ~8 s). The native platform builds and runs the validator (the xpack's `main.c` shims `NVIC_*` with `SIGUSR1`). Two real port/kernel bugs had to be fixed, plus three build fixes:
 1. **ISR context.** The POSIX-arch port's `in_handler_mode()` only looked at its own `_in_isr[]`, which its tick/IPI handlers set; the validator's `SIGUSR1` handler instead bumps `signal_nesting`. `TC_ThreadInterrupts` therefore saw thread mode and got the wrong “called from ISR” return codes. The port now defines `signal_nesting` and `in_handler_mode()` returns `_in_isr[cpu] || signal_nesting != 0` (the ARM ports have no banked mode to ask either).
 2. **Deferred wake-up.** `message_queue::send()` (used by `osMessagePut` with `osWaitForever`) and its `try_send/receive/try_receive/` `timed_receive` siblings called `internal_try_*` — which resumes a waiter — *inside* their critical section. The port cannot switch while the kernel lock is held, so it deferred to the next 1 ms tick, and `TC_MsgQWait`'s immediate assert failed. Each now reschedules after the section,

exactly as the kernel's `thread::resume()` does, so the woken thread runs at once (the way PendSV does on ARM).

3. Build: the native platform-library.cmake named a stale alias (`micro-os-plus::iii-posix-arch` → `micro-os-plus::posix-arch`); it linked `libucontext` (whose sources no longer compile against the current `glibc` — the port now prefers `glibc`'s own `ucontext`), now dropped; and the kernel's `timegm.c` prototype had to be guarded (`__GLIBC__ / __APPLE__` declare `timegm`, `newlib` does not) or the strict native flags turned the redundant redeclaration into an error. `test-cmsis-os-validator-host` added to the native config.

- **The last batch of platforms — now green.** `qemu-cortex-m0 / m3` failed to build on `include/cmsis-plus/posix/dirent.h`: an empty `DIR` struct written as `{ ; }`, which `-Wextra-semi -Werror` rejects (“extra ‘;’ inside a struct”). The stray `;` became a named `int reserved;`. Both then build and pass **3/3** (`rtos-apis`, `mutex-stress`, `cmsis-os-validator`), as do `qemu-cortex-m4f / m7f`. `nucleo-f767zi`, `nucleo-h743zi` and `raspberrypi-pico` build all three suites (3 hwd each); like `nucleo-f411re` their tests drive the board through OpenOCD, so they are now labelled `hwd` (a directory-level `set_tests_properties`) and the generic test action (`ctest -LE hwd`) skips them — `test-<platform>-cmake` runs them on a bench.

hwd tests are registered but never run — they need the board, one test per power cycle. They carry `LABELS hwd`, so `ctest -LE hwd` runs only the emulated/host set.

Per-platform notes (the recipe, applied)

- `platforms/{qemu-cortex-m0,m3,m4f,m7f,nucleo-f411re,nucleo-f767zi,nucleo-h743zi,raspberrypi-pico}` — the **Cortex-M** case, now **fixed**. Those platforms do not test a sibling port: they pull the *published* `@micro-os-plus/micro-os-plus-iii-cortexm` `xpack` (v1.1.0) plus the harness's own `device-qemu-cortexm`. The link failed with undefined reference to `Reset_Handler / NMI_Handler / ...` because the kernel's startup and exception handlers are an **optional group** (`micro-os-plus::iii-startup`), not part of the core, and the platform did not link it — nor the `semihosting/newlib/posix-io` groups the startup needs. Added to every Cortex-M `platform-library.cmake`:

```
micro-os-plus::iii-startup      # Reset_Handler, core handlers,
                                startup.cpp
micro-os-plus::iii-semihosting  # os_startup_initialize_args,
                                os_terminate
micro-os-plus::iii-newlib-reent # _close, ...
micro-os-plus::iii-posix-io     # rtos-apis reaches POSIX-io
```

Verified: `qemu-cortex-m7f` and `qemu-cortex-m4f` build all three suites (`rtos-apis`, `mutex-stress`, `cmsis-os-validator`) and `mutex-stress-test` passes under QEMU.

The **local** cortexm port's own boards are wired the same way, as cortexm-<board> platforms that reuse the port's builder. cortexm-pico2 is done: **14 CTest cases** (2 -qemu + 12 -hwd) and it builds clean.

The QEMU split is **configuration**, done in the port, not in the tests:

- test/CMakeLists.txt gained a per-test BOARD_TEST_HWD_ONLY list (a test there gets no qemu image even when the board has a QEMU linker);
- test/boards/pico2/board.cmake gained UOS_BOARD_LINKER_QEMU (linker-qemu.ld, the generic Cortex-M33 memory map);
- test/pico2/tests.cmake lists the USB pair and the multicore/SIO tests HWD_ONLY, leaving exc-test, smp-test1, sc-test-ko with a qemu image.

Caveat, and where it stopped: the RP2350's own blocks (SIO, USB, PSRAM) are not modelled by QEMU's generic Cortex-M33 machine, and making the three qemu tests actually run needs more than configuration. Work done toward it:

- a QEMU console glue, test/boards/pico2/qemu/{include/bsp,src}/{uart,led}.hpp,cpp (same bsp/uart.hpp API, implemented over the kernel's semihosting trace);
- per-variant hooks in the port's test/CMakeLists.txt: board_test_qemu_libs/_sources/_includes let the qemu image replace the board entirely;
- the pico2 tests.cmake uses them to link the kernel + the harness's device-qemu-cortexm (its vectors-cortexm.c owns the vector table) instead of the board;
- the cortexm-pico2 platform adds device-qemu-cortexm and the arm-cmsis xpack, and the config inherits arm-cmsis-dependencies.

The remaining blocker: the port's **core** (src/rtos/os-core.cpp / os-core-rp2350.cpp) is RP2350-specific and is what provides cmsis-plus/rtos/port/os-decls.h; the qemu image links the kernel, not the board, so that header is missing (fatal error: cmsis-plus/rtos/port/os-decls.h: No such file). Running the emulated pico2 tests therefore needs a **generic Cortex-M33 port core** (a port refactor), not a configuration change.

Progress on that refactor (all done, build is green):

- a generic port library micro-os-plus::cortexm-qemu in the port (the generic core src/rtos/os-core.cpp + the generic include/), with Cortex-M7 flags;
- the emulated image targets QEMU's generic **Cortex-M7** machine (mps2-an500) - the harness device's CMSIS core has no Cortex-M33;

- exc-test (a bare-metal probe that defines its own PendSV_Handler / SysTick_Handler) is HWD_ONLY: it clashes with the device;
- the emulated image takes the **device's** linker script, not the board's (_variant_linker is cleared when the board is replaced).

State now: cortexm-pico2 **builds and runs green** with 2 qemu tests (sc-test-ko-qemu, smp-test1-qemu) + 12 hwd.

The runtime “hang” had two causes, both fixed:

- **no output** — the kernel's src/diag/trace-semihosting.cpp only compiles under OS_USE_TRACE_SEMIHOSTING_DEBUG / _STDOUT; without it the weak no-op trace::write won and nothing was printed. The generic micro-os-plus::cortexm-qemu library now defines OS_USE_TRACE_SEMIHOSTING_STDOUT on its interface.
- **never exiting** — the pico2 tests print RESULT: and then loop forever (a hardware convention), so a plain qemu CTest case always timed out. Added the port's test/pico2/include/hw_result.hpp: under QEMU_BUILD the test ends through the strong semihosting _Exit() (src/semihosting-exit.cpp) and CTest reads the exit code; on hardware it is a no-op (the pico2 hw.sh does not enable semihosting, so a hardware _Exit() would halt on an unhandled BKPT) and the LED/heartbeat loop runs. sc-test-ko calls it after its RESULT line; smp-test1 gained a bounded ten-beat self-check + RESULT before it.

Both now pass in a few seconds: cortexm-pico2-sc-test-ko-qemu ... Passed 0.70 sec, cortexm-pico2-smp-test1-qemu ... Passed 3.24 sec.

Hardware tests now end with a verdict too. The same hw_result helper applies to the hwd builds: every test that prints a RESULT line calls hw_result::ok()/fail() after it, so the run stops instead of looping.

- test/pico2/include/hw_result.hpp now always ends through the strong semihosting _Exit() (both qemu and hwd; SEMIHOST is defined for every build of this port).
- test/boards/pico2/hw.sh and test/boards/pico2-pizero/hw.sh enable semihosting on both cores before the resume and keep OpenOCD attached, reading the verdict from the log (RESULT: PASS|FAIL) instead of shutting down after the resume. Without semihosting the exit BKPT is an unhandled debug event and the core halts.
- smp-mat-test, smp-test4, smp-test5 (after a TX-ring drain) and smp-test-ko gained the call; sc-test-ko and smp-test1 already had it. The probe tests without a verdict (exc-test, smp-test0, smp-test2, smp-test3, the USB pair) keep their idle loops.

cortexm-pico2-pizero — done (hardware only). The Pi-Zero RP2350B is the pico2 board plus pins/flash/probe and one real difference: it is built `__ARM_ARCH_8M_MAIN__`, and QEMU's generic Cortex-M machine has no Cortex-M33. So the board's `board.cmake` clears `UOS_BOARD_LINKER_QEMU` and the port's builder emits `hwd` alone; the platform registers 10 `-hwd` cases. Builds green; `xpm run test` finds nothing (no emulated set), as intended.

cortexm-pico2-rp2350b-psram — done. The WeAct RP2350B is the pico2 board plus pins/flash/PSRAM, on the same silicon (ARMv7E-M), so it keeps a `qemu` suite for the two emulatable tests (`sc-test-ko`, `smp-test1`) and lists the rest `HWD_ONLY`: the multicore tests need the SIO block QEMU does not model, and the PSRAM / nested-interrupt / clock-tree tests need the board's own memory map and peripherals. 16 cases (2 `qemu` + 14 `hwd`); builds green, both emulated tests pass.

The board test directories are independent copies, so the `hw_result` change was applied to the Pi-Zero and PSRAM copies too, and `hw_result.hpp` moved from `test/pico2/include/` to `test/boards/shared/` – the one include directory every board's tests now get (`test/CMakeLists.txt`), so a new board does not need its own copy.

cortexm-nucleof411, cortexm-weactf411, cortexm-weactf412 — done (hardware only). The three STM32F4 boards are programmed over their ST-Link and run on the desk, so each board declares no `UOS_BOARD_LINKER_QEMU` and the port's builder emits `hwd` alone. The platforms register what it produced:

- `cortexm-nucleof411` — 1 case (`mos-test1`).
- `cortexm-weactf411` — 2 cases (`mos-test1`, `spi-pipeline`).
- `cortexm-weactf412` — 2 cases (`mos-test1`, `uart-test1`). All build green. The `hw_result` verdict was added to their tests too: `spi-pipeline` prints its own PASS/FAIL and now ends with `RESULT:/hw_result`; `mos-test1` (all three) and `uart-test1` gained a bounded five-second run, a `RESULT` line and the semihosting exit. Their `hw.sh` already enabled semihosting, and now stays attached and reads the verdict instead of leaving OpenOCD running.

The Cortex-M family is now complete: `cortexm-pico2` (2 `qemu` + 12 `hwd`), `cortexm-pico2-pizero` (10 `hwd`), `cortexm-pico2-rp2350b-psram` (2 `qemu` + 14 `hwd`), `cortexm-nucleof411` (1 `hwd`), `cortexm-weactf411` (2 `hwd`), `cortexm-weactf412` (2 `hwd`).

- the harness suites, now all wired: `rtos-apis` (needs `xpacks::chan-fatfs` + `micro-os-plus::iii-posix-io` + its own `os_main`), `mutex-stress`, `cmsis-os-validator` (native 60/60, Cortex-M QEMU 3/3, hardware on `nucleo-*`), instrumentation and `blinky` (hardware, `nucleo-f411re`); and the AArch64 platform-support `_Exit()` fix above.

VS Code plugin — done. The plugin lists xpm **actions**, not CTest tests. Every configuration has a generic test that runs the whole emulated/host set (ctest -V -LE hwd), so the hardware cases are never run by it. The emulated/host platforms additionally have one action per test, test-<app>-<variant> (e.g. test-smp_test0-qemu, test-mutex-stress-host, test-cmsis-os-validator-host): native, aarch32-rpi-zero-2w, aarch32-luckfox-lyra, aarch64-rpi-zero-2w, and the six cortexm-* platforms. The upstream hardware platforms (qemu-cortex-m*, nucleo-*, raspberrypi-pico) have only the generic test plus a top-level test-<platform>-cmake that runs **all** their cases with a bare ctest -V (needed because their tests are all hwd, which the generic test skips). Each action configures with the pinned toolchain ({{ properties.commandCMakePrepareWithToolchain }}, not the bare commandCMakeReconfigure) so a fresh tree does not pick the host compiler.

Verified: xpm run test-smp_test0-host --config native-cmake-gcc-debug → native-smp_test0-host ... Passed; and xpm run test-sc-test-ko-qemu --config cortexm-pico2-cmake-gcc-debug → cortexm-pico2-sc-test-ko-qemu ... Passed.

Cortex-M33 QEMU platforms (pico2-1cpu & 2xcortex-m33) & SMP cmsis-os-validator — 100% green. * pico2-1cpu (QEMU mps2-an505, 1x Cortex-M33): 3/3 tests pass (rtos-apis, mutex-stress, cmsis-os-validator 60/60). * 2xcortex-m33 (QEMU mps2-an521, 2x Cortex-M33 SMP): 3/3 tests pass (rtos-apis, mutex-stress, cmsis-os-validator 60/60). * **SMP fixes for cmsis-os-validator: 1. Running thread re-enqueue guard:** thread::resume() only enqueues threads whose state is state::suspended or state::initializing. On SMP, a running thread has ready_node_.next() == nullptr; previously, raising signals/flags (flags_raise()) invoked resume() which linked the actively executing thread back into ready_threads_list_, allowing the secondary core to pick and execute the same thread concurrently on the identical stack. 2. **Thread attributes CPU affinity:** Added th_cpu_affinity to thread::attributes and os_thread_attr_t under OS_USE_SMP_SCHEDULER, passed into thread::thread constructor to ensure threads are created with affinity before internal_construct_ / resume() places them on the ready list. 3. **CMSIS-RTOS single-core compatibility:** CMSIS-RTOS v1 is fundamentally single-core; osThreadCreate and os_main_thread pin threads to Core 0 (1u <= 0). This ensures strict priority preemption (TC_ThreadPriorityExec and TC_MutexPriorityInversion) and guarantees that per-core private NVIC registers on Cortex-M handle test interrupts on the core that enabled them (TC_ThreadInterrupts). 4. **SMP stack pointer invariant:** In switch_stacks(sp) (os-core-m33.cpp and os-core-rp2350.cpp), when a core continues executing the same thread (new_thread == old_thread), old_thread->context_.port_.stack_ptr = nullptr is cleared so secondary cores do not consider the live context switchable.

Multi-Architecture & SMP/1-CPU Test Verification Matrix (100% Green)

All three base test suites (mutex-stress, rtos-apis, cmsis-os-validator) have been verified across QEMU SMP and 1 CPU architectures, as well as POSIX host:

Platform / Board	Machine / CPU	Mode	mutex-stress	rtos-apis	cmsis-os-validator	Status
pico2-1cpu	QEMU MPS2 AN505 (Cortex-M33)	1 CPU	PASSED (28.3s)	PASSED (4.7s)	PASSED (60/60)	PASS
2xcortex-m33	QEMU MPS2 AN521 (Cortex-M33)	2 CPU SMP	PASSED (28.4s)	PASSED (4.7s)	PASSED (60/60)	PASS
native	POSIX synthetic SMP (Host)	1 CPU / Host	PASSED (15.2s)	PASSED (7.2s)	PASSED (60/60)	PASS
qemu-cortex-m0	QEMU MPS2 AN385 (Cortex-M0)	1 CPU	PASSED (23.7s)	PASSED (4.7s)	PASSED (2.6s)	PASS
qemu-cortex-m3	QEMU MPS2 AN385 (Cortex-M3)	1 CPU	PASSED (23.7s)	PASSED (4.7s)	PASSED (2.3s)	PASS
qemu-cortex-m4f	QEMU MPS2 AN386 (Cortex-M4F)	1 CPU	PASSED (22.9s)	PASSED (4.7s)	PASSED (2.3s)	PASS
qemu-cortex-m7f	QEMU MPS2 AN500 (Cortex-M7F)	1 CPU	PASSED (22.8s)	PASSED (4.7s)	PASSED (2.4s)	PASS
aarch32-rpi-zero-2w	QEMU raspi3b (Cortex-A53)	4 CPU SMP	PASSED (36.0s)	PASSED (7.4s)	<i>N/A (NVIC-only)</i>	PASS
	QEMU raspi3b					PASS

Platform / Board	Machine / CPU	Mode	mutex-stress	rtos-apis	cmsis-os-validator	Status
aarch64-rpi-zero-2w	(Cortex-A53)	4 CPU SMP	PASSED (35.9s)	PASSED (7.3s)	N/A (NVIC-only)	

Note on Architecture Support: * cmsis-os-validator is specific to Cortex-M NVIC (uses NVIC_EnableIRQ, IRQn_Type) and POSIX (which provides simulated signal-based NVIC shims). It is not applicable to bare-metal AArch32/AArch64 GIC architectures. * In QEMU, raspi3b requires a minimum of 4 CPUs (-smp 4) per the BCM2837 SoC definition. * device-qemu-cortexm/include/cmsis-plus/cortexm/exception-handlers.h had multi-line comment warnings (-Werror=comment) caused by trailing backslashes on single-line comments, which was fixed and committed.

Commands that work today

```
cd "$HOME/TMP/micro-os-plus-iii-smp.git/tests"

# AArch32 Pi – the whole emulated set (port tests + mutex-stress +
#   rtos-apis)
xpm run test --config aarch32-rpi-zero-2w-cmake-gcc-debug

# one suite only
xpm run test-rtos-apis-test-qemu --config aarch32-rpi-zero-2w-
cmake-gcc-debug

# POSIX-arch on the host
xpm run test --config native-cmake-gcc-debug
```

xpm run test sets the toolchain/QEMU PATH and passes -LE hwd, so the hardware cases are never run. A bare ctest -LE hwd in the build folder does the same filtering but needs build/<config>/xpacks/.bin on PATH for the emulator.

What a fresh clone gives you

The tests are **source**, and the sources are kept — but only if the commit includes the **untracked** files. A plain git commit -a would not: it commits modified *tracked* files only, and the new platforms are untracked. So commit with git add -A.

Untracked today (absent from a clone unless added):

- micro-os-plus-iii-smp.git/tests/platforms/cortexm-{pico2,pico2-pizero,pico2-rp2350b-psram,nucleof411,weactf411,weactf412}/ — the six new platforms, five files each;

- `micro-os-plus-iii-cortexm.git/test/boards/pico2/linker-qemu.ld`, `.../test/boards/pico2/qemu/{include/bsp,src}/*` (six files), and `.../test/boards/shared/hw_result.hpp`.

Already tracked, so committed with the rest: the shared runner `micro-os-plus-iii-smp.git/test_smpl/` (`run-qemu.sh`, `run-hw.sh`, `run-host.sh`, `hw_result.hpp`, `board-contract.cpp`), the harness device `tests/device-qemu-cortexm/`, the ports' own test sources, and every `tests/platforms/* CMakeLists`.

Never in the repo, by design:

- `build/<config>/` — gitignored (`build*/`); `xpm run install / prepare / build` regenerate it;
- `tests/xpacks/` and `node_modules/` — installed by `xpm install / npm install`; the QEMU, CMake, Ninja, build-helper, chan-fatfs, validator and SEGGER packages are symlinks into `~/.local/xPacks`;
- hardware: an `hwd` test is registered but needs the real board and OpenOCD.

So, from a clone to green:

```
# 1. the six repositories, beside each other, as `*.git` (the
    ports find them
#    by either name, see Step 2)
cd "$HOME/TMP"
for r in aarch32 aarch64 cortexm devices posix-arch smp; do
    git clone "$HOME/Downloads/GIT/micro-os-plus-iii-$r.git" \
        "$HOME/TMP/micro-os-plus-iii-$r.git"
done

# 2. the tools (xpacks, including QEMU)
cd micro-os-plus-iii-smp.git/tests
npm install
xpm install

# 3. one configuration, end to end -- install, prepare, build,
    test, in that
#    order; `test` alone does not build (see Step 4)
xpm run install --config aarch32-rpi-zero-2w-cmake-gcc-debug
xpm run prepare --config aarch32-rpi-zero-2w-cmake-gcc-debug
xpm run build   --config aarch32-rpi-zero-2w-cmake-gcc-debug
xpm run test    --config aarch32-rpi-zero-2w-cmake-gcc-debug #
    emulated set

# or a whole emulated family, four steps each
for c in qemu-cortex-m0 qemu-cortex-m3 qemu-cortex-m4f qemu-
    cortex-m7f; do
    C="$c-cmake-gcc-debug"
    xpm run install --config "$C" && xpm run prepare --config "$C"
        && \
    xpm run build   --config "$C" && xpm run test    --config "$C"
```


done

```
# the same set on release -- a separate build folder, so its own
install too
for c in qemu-cortex-m0 qemu-cortex-m3 qemu-cortex-m4f qemu-
cortex-m7f; do
    C="$c-cmake-gcc-release"
    xpm run install --config "$C" && xpm run prepare --config "$C"
    && \
    xpm run build --config "$C" && xpm run test --config "$C"
done
```

The qemu cases then run under QEMU; the hwd cases are listed but skipped (-LE hwd) until a board is attached.